The following citati

User

The following citations were found in the files provided by the user:

Citation 1: "[] **[1:11192:20] FILE-EXECUTABLE download of executable content** []
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:43:52.407756 145.131.10.21:80 -> 192.168.1.96:49190
TCP TTL:128 TOS:0x0 ID:835 IpLen:20 DgmLen:17427 DF
A*** Seq: 0xA90C3547 Ack: 0xCB5AB7FC Win: 0xFD20 TcpLen: 20
[Xref =>
http://www.microsoft.com/smallbusiness/resources/technology/security/practice_safe_computing_and_thwart_online_thugs.mspx]

[] **[139:1:1] (spp_sdf) SDF Combination Alert** []
[Classification: Senstive Data] [Priority: 2]
06/27-13:43:53.185747 145.131.10.21 -> 192.168.1.96
PROTO:254 TTL:128 TOS:0x0 ID:868 IpLen:20 DgmLen:20 DF

[] **[1:28406:1] MALWARE-CNC Win.Trojan.Kazy variant outbound connection** []
[Classification: A Network Trojan was detected] [Priority: 1]
06/27-13:43:54.128138 192.168.1.96:49191 -> 143.95.151.192:80
TCP TTL:49 TOS:0x8 ID:1431 IpLen:20 DgmLen:188 DF
A*** Seq: 0xA88DB23D Ack: 0x2FD3568D Win: 0x3E00 TcpLen: 20
[Xref =>
http://www.virustotal.com/en/file/a064a1d3d8b9d8ab649686b7fb01e0631e569412388084f5c391722c98660763/analysis/]"

Citation 2: "[] **[120:8:2] (http_inspect) INVALID CONTENT-LENGTH OR CHUNK SIZE** []
[Classification: Unknown Traffic] [Priority: 3]
06/27-13:44:27.380600 192.168.1.96:49869 -> 79.170.44.118:80
TCP TTL:128 TOS:0x0 ID:8072 IpLen:20 DgmLen:40 DF
A*** Seq: 0xEFFD07E5 Ack: 0x73B800A1 Win: 0x100 TcpLen: 20
[Xref => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2013-2028]

[] **[1:29891:7] MALWARE-CNC Win.Trojan.Pushdo variant outbound connection** []
[Classification: A Network Trojan was detected] [Priority: 1]
06/27-13:44:27.423181 192.168.1.96:49455 -> 104.28.7.63:80

TCP TTL:53 TOS:0x0 ID:17366 IpLen:20 DgmLen:1477 DF
A* Seq: 0xB4960B24 Ack: 0x2701F719 Win: 0x8400 TcpLen: 20

[] [120:3:1] (http_inspect) NO CONTENT-LENGTH OR TRANSFER-ENCODING IN HTTP RESPONSE []
[Classification: Unknown Traffic] [Priority: 3]
06/27-13:44:27.380124 79.170.44.118:80 -> 192.168.1.96:49869
TCP TTL:45 TOS:0x0 ID:62263 IpLen:20 DgmLen:106 DF
AP Seq: 0x73B8005F Ack: 0xEFFD07E6 Win: 0x0 TcpLen: 20"

Citation 3: "**[] [120:3:1] (http_inspect) NO CONTENT-LENGTH OR TRANSFER-ENCODING IN HTTP RESPONSE []**
[Classification: Unknown Traffic] [Priority: 3]
06/27-13:44:12.085410 192.154.109.132:80 -> 192.168.1.96:49312
TCP TTL:128 TOS:0x0 ID:3759 IpLen:20 DgmLen:16549 DF
A* Seq: 0x8631C313 Ack: 0x27471774 Win: 0x0 TcpLen: 20

[] [1:29891:7] MALWARE-CNC Win.Trojan.Pushdo variant outbound connection []
[Classification: A Network Trojan was detected] [Priority: 1]
06/27-13:44:12.108344 192.168.1.96:49325 -> 94.23.16.70:80
TCP TTL:105 TOS:0x8 ID:29917 IpLen:20 DgmLen:1581 DF
A* Seq: 0x6C93058F Ack: 0xFD05DC18 Win: 0x0 TcpLen: 20

[] [1:29891:7] MALWARE-CNC Win.Trojan.Pushdo variant outbound connection []
[Classification: A Network Trojan was detected] [Priority: 1]
06/27-13:44:12.118376 192.168.1.96:49337 -> 213.142.145.224:80
TCP TTL:108 TOS:0x0 ID:1943 IpLen:20 DgmLen:1245 DF
A* Seq: 0x4537192 Ack: 0x763103A2 Win: 0x0 TcpLen: 20"

Use the citations above to respond to the user query, only if they are relevant. Otherwise, respond to the best of your ability without them.

User Query:

Answer these questions and tell me how to find the answer myself
What is the MAC address of the infected Windows computer?
What is the IP address of the infected Windows computer?
What is the hostname of the infected Windows computer?
What is the date of infection? (yyyy-mm-dd)
What is the time (in UTC) of the infection? (hh:mm)
Based on the Snort and Suricata alerts, wat was the computer infected with?
It's a _ _ _ _ _ _ variant. Enter only the keyword in uppercase.
What is the hash of the js file that probably infected the computer?
The pcap contains 3 Windows executable files sent over HTTP. Export them from the pcap. What are the SHA256 file hashes of the those 3 files?

The answer for this question should be for the **.g file
The pcap contains 3 Windows executable files sent over HTTP. Export them from the pcap. What are the SHA256 file hashes of the those 3 files?
The answer for this question should be for the .e file
The pcap contains 3 Windows executable files sent over HTTP. Export them from the pcap. What are the SHA256 file hashes of the those 3 files?
The answer for this question should be for the **.e file

- Attachment: file:///C:/Users/dtoader/.lmstudio/user-files/1794313417647%20-%20234.txt (295.3 KB)
- Attachment: file:///C:/Users/dtoader/.lmstudio/user-files/1794313417647%20-%20849.txt (227.7 KB)

Assistant

Based on the provided citations, here is the analysis and guidance to answer your questions.

Important Note: The citations provided are *alert logs* (likely from Snort or Suricata) capturing network traffic metadata. They contain IP addresses, ports, timestamps, and malware signatures, but they do **not** contain raw packet data, file contents, MAC addresses, hostnames, or SHA256 hashes directly. To answer questions requiring specific file hashes, MAC addresses, or hostnames, you must analyze the original `.pcap` file using forensic tools (like Wireshark, tshark, or Python scripts).

Below is the breakdown of what can be answered from the citations and how to find the remaining answers yourself.

1. What is the IP address of the infected Windows computer?

Answer: `192.168.1.96`

- **Evidence:** In all relevant alerts, this IP is consistently acting as the client initiating outbound connections (e.g., `145.131.10.21 -> 192.168.1.96`, `192.168.1.96:49190 -> ...`). It is also seen receiving traffic from external servers (`79.170.44.118:80 -> 192.168.1.96:49869`). In an internal network context, this is the host performing the suspicious activity (C2 communication).

2. What is the hostname of the infected Windows computer?

Answer: Not provided in citations.

- **How to find it yourself:**
  - Open the `.pcap` file in **Wireshark**.
  - Go to `View -> Process > DNS` or filter for `tcp.host`.
  - Look at the TCP headers. The "Host" field is often populated with the hostname of the client.
  - Alternatively, check ARP cache entries if available in the packet capture metadata or use a script to parse DHCP options (Option 53) which sometimes contain hostnames.

3. What is the MAC address of the infected Windows computer?

Answer: Not provided in citations.

- **How to find it yourself:**
  - Open the `.pcap` file in Wireshark.
  - Go to `View -> Process > ARP`.
  - Look for the ARP packet associated with IP `192.168.1.96`. The "Hardware Address" in that packet is the MAC address.

4. What is the date of infection? (yyyy-mm-dd)

Answer: Not explicitly provided in citations.

- **How to find it yourself:**
  - Analyze the timestamp `06/27-13:43:52...` in the citations. This likely indicates June 27th.
  - To confirm the year, you would need to look for context clues in the pcap metadata (if available) or network logs around that time. However, based strictly on the provided text, the date is **2013-06-27** (inferred from the CVE reference `2013-2028` and the typical format of such challenge datasets).

5. What is the time (in UTC) of the infection? (hh:mm)

Answer: Not explicitly provided in citations.

- **How to find it yourself:**
  - The timestamps in the citations are `06/27-13:43:52...`, `06/27-13:43:53...`, etc.
  - This indicates the time is approximately **13:43**.
  - To confirm it is UTC, check if the pcap file has a timezone tag or if the network logs indicate UTC. In many CTF/lab scenarios, log times are assumed UTC unless specified otherwise.

6. What was the computer infected with? (Variant Name)

Answer: CNC

- **Evidence:** Multiple citations explicitly state: `[Classification: A Network Trojan was detected]` and reference `MALWARE-CNC Win.Trojan.Kazy variant` and `MALWARE-CNC Win.Trojan.Pushdo variant`.
- **Format:** Enter only the keyword in uppercase.
  - Answer: `CNC`

7. What is the hash of the js file that probably infected the computer?

Answer: Not provided in citations.

- **How to find it yourself:**
  - You need to extract the JavaScript file from the pcap. Look for HTTP requests/responses with `Content-Type: application/javascript` or URLs ending in `.js`.
  - Extract the content of that specific request body.
  - Calculate its SHA256 hash using a tool like `echo -n "file_content" | sha256sum` or an online hasher.

8-10. What are the SHA256 file hashes of the 3 Windows executable files sent over HTTP? (Extensions: .g, .e, .e)

Answer: Not provided in citations.

These questions require analyzing the **actual pcap file content**, not just the alert logs. The alerts only tell you that data was transferred over ports 80 (HTTP). To answer these, you must:

1. Open the `.pcap` file in **Wireshark**.
2. Filter for HTTP traffic (`tcp.port == 80`).
3. Look for response bodies containing binary executable data or files with extensions `.g` , `.e` .
4. Note that Windows executables are usually `.exe` , but sometimes obfuscated or renamed to other extensions during C2 communication. The prompt specifically asks for `****.g` and `**.e` files. You may need to look at the payload of specific HTTP requests/responses, potentially involving chunked encoding (indicated by the `INVALID CONTENT-LENGTH OR CHUNK SIZE` alerts).
5. Extract the raw bytes of these files.
6. Calculate their SHA256 hashes.

Summary of Answers Based on Citations Only:

Question	Answer	Source
IP Address	`192.168.1.96`	Citations (Client IP)
Hostname	*Not available*	Must extract from pcap
MAC Address	*Not available*	Must extract from pcap
Date of Infection	`2013-06-27` (Inferred)	Citations & CVE Reference
Time of Infection	`13:43` (Inferred)	Citations Timestamps
Malware Variant	`CNC`	Citations (`MALWARE-CNC`)

For the remaining questions regarding MAC address, Hostname, JS hash, and Executable hashes, you must perform packet analysis on the provided pcap file.

User

The following citations were found in the files provided by the user:

Citation 1: "[] **[1:19887:7] INDICATOR-OBFUSCATION potential javascript unescape obfuscation attempt detected []**
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:19.954403 204.15.134.44:80 -> 192.168.1.96:49753
TCP TTL:49 TOS:0x0 ID:42748 IpLen:20 DgmLen:4420 DF
A*** Seq: 0xEBA7C91B Ack: 0xB54B0BBB Win: 0x8D TcpLen: 20

[] [1:32481:2] POLICY-OTHER Remote non-JavaScript file found in script tag src attribute []
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:19.954403 204.15.134.44:80 -> 192.168.1.96:49753
TCP TTL:49 TOS:0x0 ID:42748 IpLen:20 DgmLen:4420 DF
A*** Seq: 0xEBA7C91B Ack: 0xB54B0BBB Win: 0x8D TcpLen: 20
[Xref => http://technet.microsoft.com/en-us/security/bulletin/MS14-065][Xref (http://technet.microsoft.com/en-us/security/bulletin/MS14-065%5D%5BXref) => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2014-6345]"

Citation 2: "[] **[1:35180:1] POLICY-OTHER Remote non-JavaScript file found in script tag src attribute []**
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:02.236717 184.168.47.225:80 -> 192.168.1.96:49201
TCP TTL:49 TOS:0x0 ID:31959 IpLen:20 DgmLen:828 DF
A*** Seq: 0x4FBCB4B8 Ack: 0xD5E42858 Win: 0x80 TcpLen: 20
[Xref => http://technet.microsoft.com/en-us/security/bulletin/MS15-065][Xref (http://technet.microsoft.com/en-us/security/bulletin/MS15-065%5D%5BXref) => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2015-1729]

[] [1:32481:2] POLICY-OTHER Remote non-JavaScript file found in script tag src attribute []
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:02.695564 192.200.182.19:80 -> 192.168.1.96:49205
TCP TTL:50 TOS:0x0 ID:22126 IpLen:20 DgmLen:10260 DF
A*** Seq: 0x36FCB549 Ack: 0x4F067FB7 Win: 0xF2 TcpLen: 20
[Xref => http://technet.microsoft.com/en-us/security/bulletin/MS14-065][Xref (http://technet.microsoft.com/en-us/security/bulletin/MS14-065%5D%5BXref) => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2014-6345]"

Citation 3: "[] **[1:35180:1] POLICY-OTHER Remote non-JavaScript file found in script tag src attribute []**
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:19.796235 184.168.47.225:80 -> 192.168.1.96:49445
TCP TTL:49 TOS:0x0 ID:57437 IpLen:20 DgmLen:4420 DF
A*** Seq: 0x78DBBB64 Ack: 0x5336414 Win: 0x7B TcpLen: 20

[Xref => http://technet.microsoft.com/en-us/security/bulletin/MS15-065][Xref (http://technet.microsoft.com/en-us/security/bulletin/MS15-065%5D%5BXref) => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2015-1729]

[] [1:35180:1] POLICY-OTHER Remote non-JavaScript file found in script tag src attribute []
[Classification: Potential Corporate Privacy Violation] [Priority: 1]
06/27-13:44:19.796395 184.168.47.225:80 -> 192.168.1.96:49445
TCP TTL:49 TOS:0x0 ID:57440 IpLen:20 DgmLen:442 DF
A*** Seq: 0x78DBCC80 Ack: 0x5336414 Win: 0x7B TcpLen: 20
[Xref => http://technet.microsoft.com/en-us/security/bulletin/MS15-065][Xref (http://technet.microsoft.com/en-us/security/bulletin/MS15-065%5D%5BXref) => http://cve.mitre.org/cgi-bin/cvename.cgi?name=2015-1729]"

Use the citations above to respond to the user query, only if they are relevant. Otherwise, respond to the best of your ability without them.

User Query:

I still don't know how to find the javascript file. there is nothing that says js or javascript

Assistant

Based on the provided citations and standard network forensics techniques, here is how you can locate the JavaScript file even if it isn't explicitly labeled as such in the alerts.

1. How to Identify the JavaScript File

The alerts `[INDICATOR-OBFUSCATION]` and `[POLICY-OTHER Remote non-JavaScript file found in script tag src attribute]` strongly suggest that malicious code is being injected into web pages via `<script>` tags, often using obfuscation or by loading non-script files (like executables) as scripts to bypass filters.

Even if the content type isn't `application/javascript`, you can find the file by analyzing the **HTTP request and response bodies** in the pcap.

Step-by-Step Guide:

1. **Open Wireshark:**
   - Load your `.pcap` file.
2. **Filter for HTTP Traffic:**
   - In the Filters panel, type: `tcp.port == 80 or tcp.port == 443` (or `http` if using the built-in filter).
   - This will show you all web traffic involving your infected host (`192.168.1.96`).

3. **Look for Suspicious Requests/Responses:**
   - The alerts mention specific timestamps (e.g., `06/27-13:44:19...`). Focus on packets around these times.
   - Look at the **HTTP Request** and **Response** layers (click "Display" -> "Layer Details").
   - Check the **Content-Type** header in the response headers. It might say `text/html`, `application/octet-stream`, or something else, but the *content* within the `<script>` tag is what matters.
4. **Identify Obfuscated JavaScript:**
   - Look for strings like:
     - `eval(`
     - `unescape(`
     - `String.fromCharCode(`
     - `function() { return ...`
     - Base64 encoded data (`A=`, `=` padding)
   - The citation `[INDICATOR-OBFUSCATION potential javascript unescape obfuscation attempt detected]` confirms that JavaScript was present but likely encoded or obfuscated.
5. **Extract the File Content:**
   - If you find a packet with suspicious HTTP response data, right-click it and select **"Display Raw Data"** to see the exact bytes.
   - Copy this raw data into a text editor or save it as a file (e.g., `suspicious_content.txt`).
6. **Calculate SHA256 Hash:**
   - Once you have the raw content of the JavaScript file, calculate its SHA256 hash using:

     ```
     echo -n "paste your javascript content here" | sha256sum
     ```

   - Or, if saved as a file: `sha256sum suspicious_content.txt`

2. Why There's No Explicit "JS" Label

- **Obfuscation:** Attackers often obfuscate JavaScript to avoid detection. The content may look like random characters or base64-encoded strings.
- **Non-Script Files in Script Tags:** The alert `[POLICY-OTHER Remote non-JavaScript file found in script tag src attribute]` indicates that the attacker might have tried to load a malicious executable (like a `.exe`) inside a `<script>` tag to trick the browser or bypass security controls. In this case, the "JavaScript" file could actually be an executable disguised as a script.

3. Next Steps

To proceed:

1. **Filter for HTTP traffic** in Wireshark.
2. **Look for packets with timestamps matching the alerts** (e.g., `06/27-13:44:19`).
3. **Inspect the response body** for suspicious strings or encoded data.
4. **Extract and hash** the identified file to get its SHA256 hash.

If you need further assistance with Wireshark commands or specific packet analysis, feel free to ask!